Bay Area DZ

Getting started

Claim $50 in OpenAI credits
Every competitor gets them to build their bot.

Claim your credits

Quick start for your team build box

We're providing an EC2 instance that is similar to the tournament runtime environment so you can test your code in a realistic compute environment.

Use the skill | Do it myself

You can have an AI coding assistant such as Codex or Claude Code do the setup: it registers your SSH key and gets your build machine and the kit ready, asking for an API key from your Settings.

`https://pokerbot.sequoiacap.com/tournament/no-limit-2026-setup/SKILL.md`

Copy the address and ask your assistant to download the skill from it and follow it. There is nothing to download yourself, unless you would rather save the file to your assistant's skills folder.

Run the setup on a smaller model. It is routine work, and a frontier model such as Astra or Fable burns through your budget before your bot plays a hand.

— Downloaded the starter kit

✓ Generated an API key view it

✓ Added a public key view your keys

Each bot is a Python **Agent** with one method: **act**. Each round snapshots every team's active bot, and scored rounds play it at thousands of 8-player sit-and-go tables; its round score is its average points per table.

Skip navigation

Build and submit

1 Download the starter kit

To get started, grab our starter kit. Here's what's inside:

agent.py	A template bot that plays legally, with a memory example.
examples/	**check_or_fold** and **min_raiser**, opponents for local play.
sdk/	The protocol, the engine with a local simulator, the ICM calculator, and the **poker-agent** command.
requirements.txt	The exact libraries and versions the competition machines install.
RULES.md	The announced variant's rules.
hand-history/	A sample in the participant download format, and a streaming reader.
README.md	Setup, local testing, submission, and the hand-history format.
AGENTS.md, **CLAUDE.md**	Instructions for AI coding assistants.

You'll be able to grab the starter kit and upload bots when the organizers announce the variant at the start of the event.

2 Run it locally

To run some test rounds locally, enter the kit folder:

```
python3.12 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pip install -e ./sdk/agent-protocol -e ./sdk/engine -e ./sdk/agent-sdk
```

Skip navigation

```
poker-agent check agent.py
poker-agent play . examples/check_or_fold examples/min_raiser --tables 200
```

check FILE runs the server's upload checks on one file or zip without executing code. In a zip it parses every **.py** file and rejects the upload if any of them imports **poker_engine** or **pokerkit**, even one the bot never loads; **check agent.py** sees only **agent.py**, while **pack** checks the whole zip it builds.

play runs sit-and-go tables of the variant in **RULES.md** with one seat per folder argument, 2 to 10. The command above plays 3-handed tables, where every place pays; repeat folders to fill the 8 seats a round uses. Each folder runs as two processes shared by its seats. **play** prints the last 64 KB of each process's stderr to stderr, then reports average points, finishing places and failed decisions per seat as JSON on stdout. **--seed** replays the same randomness, and changes under **sdk/** affect only local play.

3 Write act

```
class Agent(poker_agent_sdk.Agent):
    def act(self, state, memory):
        # Return an Action and bytes to remember.
        return action, memory
```

To have your bot actually do something, return an **Action** and the bytes you want back as **memory** on your next turn at this table. Betting actions are **Action.fold()**, **Action.check()**, **Action.call()** and **Action.raise_to(total)**, where **total** is your whole bet for the current betting round. Showdown hands are revealed automatically, without a bot decision. A variant may add action kinds; its rules describe how to build them.

state.can(kind) returns that kind's legal action, with **min_amount** and **max_amount**, or **None**. In reference hold'em **state.phase** is **"betting"**; a variant may add phases and action kinds, so choose from **state.legal_actions**. Where neither check nor fold is offered, **Action(first.kind, first.min_amount)** with **first = state.legal_actions[0]** takes the automatic choice.

4 Upload it to your team

Upload **agent.py**, or a zip with **agent.py** at its root plus the files it reads. To s process, run **poker-agent pack DIR OUT.zip**, which zips **DIR**

reproducibly, leaving out hidden files and folders and **__pycache__**, refusing symlinked files, skipping symlinked folders, and running **check** on the result, so **DIR** should hold only your bot, not the kit root. There are size limits on the package you can upload:

- 50 MB uploaded
- 200 MB unpacked
- 1,000 archive entries (i.e., files and folders in the zip file)
- 1 MB for the code in **agent.py**
- Paths of up to 512 characters and 16 levels

Your team can upload once every 30 seconds.

Every bot that is uploaded is put through a series of checks to make sure the bot is a viable player. Each upload first plays a heads-up test match of up to 20 hands against a bot that checks when free and otherwise folds, in the competition's variant with the default 1 ms decision limit and no time bank, whatever the rounds use. It is rejected if it does not start within the startup allowance (the rejection states the limit), crashes or stops responding (it is not restarted), or fails more than 10% of its decisions. **play** always gives a 100 ms bank, so a bot that needs the bank can pass locally and still be rejected. A rejected upload shows its reason and validation log on your team page.

Go to your team

5 Make it active

Each upload that passes its test match becomes active automatically, unless the competition is frozen or has ended when the match finishes; that upload then stays ready until you choose it. To play an earlier upload instead, you can choose it on your team page. Each round snapshots every team's active bot when it starts, so changes apply from the next round, and a team without an active bot forfeits that round with zero points. If the organizers freeze the competition, uploads and active-bot changes are refused.

⚠ What will break your bot

Skip navigation

Decision time	**state.config.decision_ms** per decision (1 ms by default); overruns draw on a per-table bank of **state.config.time_bank_ms** (100 ms by default). The upload test match has no bank.
State	**self** and globals cannot be relied on between calls. Per-table state travels in **memory**, truncated at **state.config.memory_bytes** (64 KB by default), so parse it defensively; carrying information between tables breaks the rules.
Libraries	Python 3.12, the standard library, **poker_agent_sdk**, **poker_agent_protocol** and exactly the packages in **requirements.txt**: NumPy, SciPy, pandas, scikit-learn, PyTorch (CPU), LightGBM, XGBoost, ONNX Runtime, Numba and eval7. **poker_engine** and **pokerkit** exist only locally.
Sandbox	No network or GPU. The bot's directory is read-only and not executable, **/tmp** is the only writable place, and OpenMP, MKL and OpenBLAS are limited to one thread.
Failed decisions	A timeout, exception or illegal action, including a raise outside the legal range, plays the fallback: check if free, otherwise fold, or the automatic choice in a phase a variant adds. Showdowns do not ask the bot for a decision.

How your bot runs

- The clock starts when the host sends a request: a decision must return within **state.config.decision_ms** plus a small delivery allowance. Time beyond that comes from the table's bank, and **state.time_bank_ms** is what remains. A timeout empties the bank; the next table starts with a full one.
- Each team runs as a pool of identical copies, and any copy may answer any table. In production the bot's microVM imports **agent.py** once and forks every copy from that process, then each copy constructs **Agent()**. Load models at import and run one throwaway prediction there: importing has its own startup allowance, while a cold first call in **act** can exhaust the bank. Only the importing thread survives a fork, so threads and thread pools started at import do not exist in the copies.

- OpenMP, MKL and OpenBLAS are limited to one thread in production, and **play** starts bots with a clean environment, so to time **act** under the same limits, set **OMP_NUM_THREADS**, **MKL_NUM_THREADS** and **OPENBLAS_NUM_THREADS** to 1 in **agent.py** before importing NumPy or PyTorch. ONNX Runtime and Numba ignore these and size their thread pools to the machine's cores, so also create ONNX Runtime sessions with **SessionOptions.intra_op_num_threads = 1** and set **NUMBA_NUM_THREADS** to 1 before importing Numba, so production runs them the same way. On macOS, NumPy and SciPy use Accelerate, which follows **VECLIB_MAXIMUM_THREADS** instead.
- Memory starts as **b""** at every table and travels inside the decision time; up to about 16 KB costs nothing measurable. After a timeout, exception or crash, the previous memory is kept and that turn's events are delivered again; after an illegal action, the memory returned with it is stored. Memory beyond the limit is truncated, and memory that makes **act** raise comes back unchanged on every later turn at that table.
- Only offered kinds are legal, and a **raise_to** total outside **min_amount** to **max_amount** is not clamped. Check and call are never offered together, and reference hold'em offers no fold when checking is free. Integers, NumPy's included, pass unchanged; floats, NumPy's included, are rounded down, so a computed size can land one chip under the minimum. Returning a kind that is not offered, or an out-of-range total, is a failed decision even when the fallback has the same effect: an undersized raise facing a bet folds.
- A copy that crashes or exits, keeps stalling its other tables with late replies, stays stuck, or breaks the protocol is killed and restarted on its next request; an exception in **act** fails only that decision. Each copy has a limited number of starts per round, and once every copy is given up on, the bot's remaining decisions on that machine play the fallback for the rest of the round.
- The bot's directory is mounted read-only and without execute permission, so compiled extensions or executables in an upload cannot load or run. **/tmp** is a 64 MB scratch space, also without execute permission, shared by the team's copies. There is no network or GPU, and bots may not keep files between tables or compute between decisions.
- Your team page shows each round's decisions, missed decisions, restarts and latest error reports: tracebacks, why a copy was killed, and a note when memory was cut. It never shows what the bot prints.

Skip navigation

- **state.events** is everything public at the table since your last turn there; on your first turn at a table, everything since it started. Each event has **kind**, **seat** (**None** for **hand** and **board**), **amount** and **cards**. In reference hold'em the kinds are **hand** (amount: the hand number), **ante** and **blind** (the chips posted), **fold**, **check**, **call** (the chips added), **raise** (the raiser's total bet for the betting round), **board** (only the newly dealt cards), **show** (the automatically revealed cards), **win** (the chips awarded; a hand can have several) and **stack** (the seat's stack when the hand ends). A variant can add kinds, so ignore the ones you do not handle.
- The hand: **seat** (yours at this table), **cards**, **board**, **pot**, **to_call**, **legal_actions**, **history** (this hand only) and **phase**. Cards are strings such as **"As"** and **"Td"**. **state.variant** holds any public state the variant adds.
- The table: **seats**, each with **seat**, **stack**, **bet**, **status** (**active**, **folded**, **all_in** or **busted**) and **shown_cards**, and **button**. Seat numbers are anonymized per table.
- The tournament: **blind_level**, **blinds**, **hands_until_next_level**, **hand_number**, **players_remaining** and **bust_order**.
- The table's fixed rules, in **state.config**: **table_id**, **seat**, **variant** (its ID), **players**, **starting_stack**, the **blinds** schedule, **hands_per_level**, **hand_cap**, **payouts**, **decision_ms**, **time_bank_ms** and **memory_bytes**. A round may draw payouts per table, so read **payouts** rather than assuming 50/30/20.
- Every player who reaches showdown automatically reveals all of their hole cards, including losing hands. Mucking is not allowed, and **act** is not called for showdown. A fold reveals no additional cards; a winner whose opponents all folded does not have to show. Cards already public stay public. The **show** events arrive on your next turn at the table and appear in completed hand histories.

Hand histories

When a round completes, your team page and the round page offer its hand history: every table your bot played, as gzipped JSON lines, with opponents named, your cards in every hand, other players' cards only when made public, and the actions played for your bot's failed decisions. You can download either the full set (which can be quite large!) or a smaller subset that may be easier to work with given the time constraints.

To help bootstrap your learning, the starter kit's **hand-history/** folder has a sample

Skip navigation hat and a streaming reader.

The variant, tables and scoring

The variant's rules, the default table settings and payouts, ICM at the hand cap, the round schedule and weights, and tiebreaks are on Format. Keep in mind, a round can change the defaults, and **state.config** holds each table's actual settings.

Connect to your team's build box

The build box is a remote Ubuntu machine set up like the ones your bot plays on in the tournament, so you can build and test your bot in an environment similar to where it will run. Each member logs in with their own SSH key from a terminal with OpenSSH: Terminal on macOS, PowerShell on Windows. Outbound port 22 must be open.

1 Have an SSH public key

macOS, in Terminal:

```
# Existing keys
ls ~/.ssh/*.pub
# Make one
ssh-keygen -t ed25519
# Copy the public key
pbcopy < ~/.ssh/id_ed25519.pub
```

Windows, in PowerShell:

```
# Existing keys
dir $env:USERPROFILE\.ssh\*.pub
# Make one
ssh-keygen -t ed25519
# Copy the public key
Get-Content $env:USERPROFILE\.ssh\id_ed25519.pub | Set-Clipboard
```

Press Enter at every **ssh-keygen** prompt: default path, no passphrase. Only the **.pub** file goes to the portal. Keys are ed25519, RSA or ECDSA; no DSA or **-sk** keys.

2 Add the key to the portal

the sidebar → Account settings → SSH keys → paste the line

Five keys per account. Each member adds their own. Live within

30 seconds.

3 Get the SSH command

Your team → Build box tab → copy the command under Connect. It is **ssh ubuntu@<address>**. "Address not available yet": the organizers have not started the fleet. "Build starts …": the box is closed until then.

4 Connect

```
ssh ubuntu@<address>
```

- First time: answer **yes** to the host-key prompt.
- Key elsewhere: **ssh -i ~/.ssh/mykey ubuntu@<address>**.
- Before the opening time: one line with the opening time (UTC), then disconnect. Your key works.
- After it: a shell as **ubuntu**. Work in **~/work**.

Optional alias, required for the Codex app. Add to **~/.ssh/config** (Windows: **notepad $env:USERPROFILE\.ssh\config**, create it if asked), then **ssh no-limit-2026-build** works:

```
Host no-limit-2026-build
  HostName <address>
  User ubuntu
  IdentityFile ~/.ssh/id_ed25519
  IdentitiesOnly yes
  ServerAliveInterval 30
```

5 Authorize Codex

On the box:

```
codex login --device-auth
```

Open the printed address in your own browser, enter the code, and sign in with your team's ChatGPT account. **codex login status** confirms it. One sign-in per t one wins.

- Codex app: Settings → Connections → add **no-limit-2026-build**, project folder **~/work**.
- No app: run **codex** in **~/work**, inside **tmux new-session -A -s no-limit-2026**.

Troubleshooting

'ssh' is not recognized (Windows)	Settings → Apps → Optional features → add OpenSSH Client. Reopen PowerShell.
Permission denied (publickey)	Wait 30 seconds after adding the key. Compare the fingerprint in Settings with **ssh-keygen -lf ~/.ssh/id_ed25519.pub**. Pass **-i** if the key is elsewhere.
Permissions … are too open	macOS: **chmod 600 ~/.ssh/id_ed25519**. Windows: **icacls "$env:USERPROFILE\.ssh\id_ed25519" /inheritance:r /grant:r "${env:USERNAME}:F"**.
doesn't start until …	Your key works and the box is closed. Return after that time (UTC).
Host key verification failed	Retry and answer **yes**. A key that changed on a box you used before: tell the organizers.
Connection timed out or **refused**	Port 22 is blocked, or the box is stopped. Try another network, then the organizers.
Maximum of 5 keys	Remove one under Settings → SSH keys.
codex: command not found	Still installing. Run **cloud-init status --wait**, then retry.
Device code refused	Your ChatGPT workspace forbids device sign-in. Ask its admin, or use SSH and tmux.

Or let an AI coding assistant do it. Install the setup skill once with **curl -fsSL --create-dirs -o ~/.agents/skills/no-limit-2026-setup/SKILL.md https://pokerbot.sequoiacap.com/tournament/no-limit-2026-setup/SKILL.md** (Claude Code: **/no-limit-2026-setup/SKILL.md**), or paste the address into your k it to download and follow it. Then ask for **$no-limit-2026-setup**

Skip navigation

(**/no-limit-2026-setup** in Claude Code) with an API key from the API keys section of your Settings.

PROTIPs

- **AGENTS.md** gives coding assistants the kit's rules and limits, and **CLAUDE.md** imports it; use the kit folder as the assistant's working directory.
- The setup skill (**no-limit-2026-setup**) is one file. It saves your API key, registers your SSH key, writes the SSH alias and connects Codex or the Claude app before the kit exists; run it again after the announcement to put the kit on the machine, or choose local-only and it downloads the kit here. A web chat alone cannot run it: use Codex, Claude Code or another agent with a terminal, and ask it to download the file rather than summarize it.
- Generated code may import only the standard library, **poker_agent_sdk** , **poker_agent_protocol** and the packages in **requirements.txt** . **check** catches only the kit's local-only packages, **poker_engine** and **pokerkit** , so test in a virtualenv with nothing else installed.
- Benchmark **act** with **time.perf_counter_ns()** over many realistic states, memory parsing included, and compare the p99 and the maximum, not the mean, with **decision_ms** .